



Université de Sherbrooke

SQL

Expressions spécialisées

SQL_03b

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

Ameni SOUID (ameni.Souid@usherbrooke.ca)

(les auteurs sont cités en ordre alphabétique nominal)

—

Scriptorum/Scriptorum/SQL_03b-Expressions-specialisees, version 1.0.0.a, en date du 2026-01-13

— document en cours de révision, ne pas citer à l'externe —

Sommaire

...

Mise en garde

Le présent document est en cours d'élaboration ; en conséquence, il est incomplet et peut contenir des erreurs.

Historique

diffusion	resp.	description
2026-01-24	CK	Récupération de notes diverses SQL03c.

Table des matières

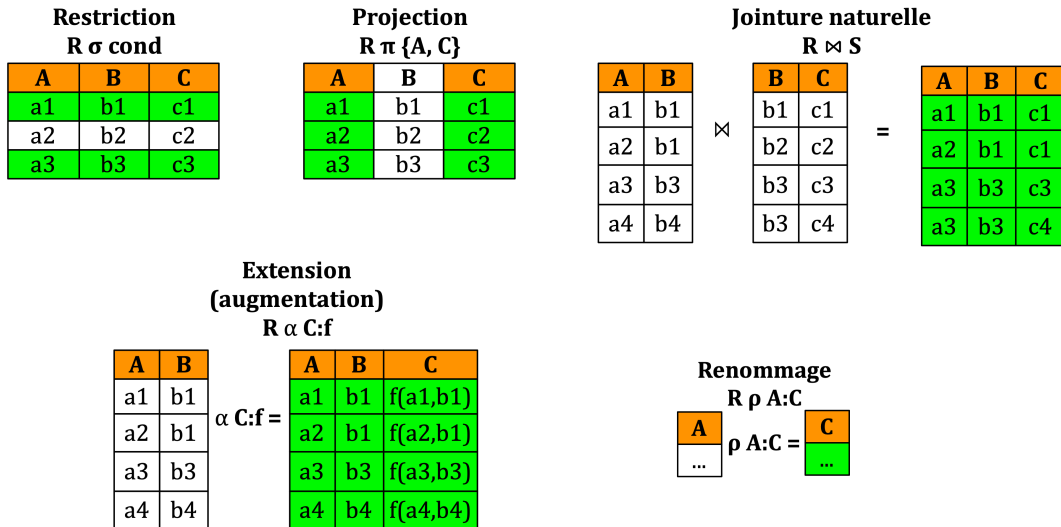
Introduction.....	4
1. Présentation.....	4
2. Agrégation.....	4
3. Groupement.....	5
4. Ordonnancement.....	9
5. Quantification.....	9
5.1. Quantification générale.....	9
5.2. Quantification de comparaison.....	10
6. Contexte.....	11
Conclusion.....	13
Références.....	14
Définitions.....	15

Introduction

Le présent document a pour but de présenter une introduction du langage de définition de requêtes qu'on retrouve dans SQL.

Dans un premier temps, nous présenterons une grammaire simplifiée, sous-ensemble propre de la grammaire de la norme ISO 9075:2016, avec, au besoin, certaines particularités de PostgreSQL.

1. Présentation



Une requête n'est pas autre chose qu'une expression SELECT.

```
requête ::=
  [ Contexte ]
  SELECT opMode { * | projection-extension }
  FROM listeDeJointures
  [ Restriction ]
  [ Groupement ]
  [ OpComplémentaire ]
  [ Ordonnancement ]
  [ Divers ]

opMode ::=
  [ DISTINCT | ALL ]
```

Le mode indique si l'expression retourne une relation (DISTINCT) ou une collection (ALL).

L'étoile indique que tous les attributs de l'expression sont retenus (sans projection).

2. Agrégation

Les fonctions d'agrégations :

- AVG(col)
- COUNT(col)
- MAX(col)
- MIN(col)
- SUM(col)
- MEDIAN(col)
- STDDEV(col)
- STDDEV_POP(col)
- STDDEV_SAMP(col)
- VARIANCE(col)
- VAR_POP(col)
- VAR_SAMP(col)

Plusieurs autres disponibles avec PostgreSQL, dont :

- string_agg
- xml_agg
- json_agg
- array_agg
- range_agg
- bit_or, bit_and

de très nombreuses fonctions statistiques...

Quand mettre ALL ou DISTINCT ?
Que se passe-t-il s'il n'y a aucun tuple ?
Comment les NULL sont-ils traités ?
Un cas spécial : COUNT(*)

Exemple — Université

- Combien y-a-t'il d'activités offertes ?

```
select count(*) as nbActivites
from Activite;
```

- Quelle est la moyenne des résultats ?

```
select avg(note) as moy_res
from Resultat;
```

- Calculer la moyenne, le minimum et le maximum des résultats des examens de l'activité IFT187.

```
select avg(note) as moy_exa
      , min(note) as min_exa
      , max(note) as max_exa
from Resultat
where te in ('IN', 'FI')
and activite = 'IFT187';
```

3. Groupement

```
Groupement ::=
[ GROUP BY { expression ... , } ]
[ HAVING { condition ... , } ]
```

Exemple — Université

- Quel est le nombre d'évaluations consigné par étudiant ?
- Quel est le nombre d'activités auxquelles un étudiant est (ou a été) inscrit ?

- Quelles sont les personnes étudiantes (ou a été) inscrites à plus que trois activités ?

- Quelle est l'activité n'ayant aucune évaluation ?

- Quel est la note totale par étudiant pour chacun de ces cours ?

4. Ordonnement

```
Ordonnement ::=
  [ ORDER BY { expression [ ordre ] [ t_null ] ... , } ]
  [ LIMIT { nombre | ALL } ] [ OFFSET nombre ]

order ::= =
  ASC | DESC | USING opérateur

t_null ::=
  NULLS { FIRST | LAST }
```

Exemple—Université

- Quelles sont les activités avec le plus d'inscriptions ?

5. Quantification

Un type de restriction dérivée de la logique du premier ordre.

```
restriction ::=
  [ WHERE condition* ]

condition ::=
  quantification_gen
  | quantification_com
  | expression
```

5.1. Quantification générale

```
quantification_gen ::=
  [NOT] EXISTS (requête)
```

$\exists x. p(x)$

ensemble non vide

$0 <> (\text{SELECT count(*) FROM ensemble})$

EXISTS (ensemble)

$\neg \exists x. p(x)$

ensemble vide

$0 = (\text{SELECT count(*) FROM ensemble})$

NOT EXISTS (ensemble)

L'opérateur de quantification complète l'équivalence avec la logique du premier ordre. Il est souvent utilisé pour les cas où il est nécessaire d'imbriquer les SELECT (par opposition aux cas où il est suffisant, et préférable, d'utiliser WITH ou GROUP BY). Le SELECT intérieur est lié au SELECT extérieur par les attributs provenant du FROM du SELECT extérieur et utilisé dans la condition du SELECT intérieur.

Sémantique

$\{x \in A \mid [\forall y \in B] (p(x,y))\}$

```
select A.x
from A
where not exists (
```

```

select 1
from B
where not p(A.x,B.y)
)

```

$\{x \in A \mid [\forall (x,y) \in B] (p(x,y))\}$

```

select A.x
from A
where not exists (
  select 1
  from B
  where A.x=B.x and not p(B.x,B.y)
)

```

$\{x \in A \mid q(x) \wedge [\forall y \in B] (p(x,y))\}$

```

select A.x
from A
where q(A.x) and not exists (
  select 1
  from B
  where and not p(A.x,B.y)
)

```

$\{x \in A \mid q(x) \wedge [\forall (x,y) \in B] (p(x,y))\}$

```

select A.x      □
from A
where q(A.x) and not exists (
  select 1
  from B
  where A.x=B.x and not p(B.x,B.y)
)

```

Exemple — Université

- Quels sont les étudiants qui n'ont aucune évaluation à ce jour ?
- Quelles sont les activités n'ayant aucun étudiant à ce jour ?

5.2. Quantification de comparaison

```

quantification_com ::=
  expression opComp [ANY | ALL] ensemble

opComp ::=
  = | <> | < | <= | > | >=

ensemble ::=
  ( requête ) | ( listeDeValeurs )

```

Exemple — Université

- Quelles sont les personnes étudiantes dont toutes les notes sont supérieures ou égales à 60 ?

```

select matricule
from Etudiant
where 60 <= all (
    select note
    from Resultat
    where Etudiant.matricule = Resultat.matricule
);

```

6. Contexte

```

Contexte ::=
    WITH [ RECURSIVE ] { requêteCont ... , }

requêteCont ::=
    nomReqCont [ ( { nomCol ... , } ) ] AS ( opDefTable )

opDefTable ::=
    requête | valeurs | insertion | miseAJour | retrait

```

Ceci est une adaptation de la notation mathématique usuelle permettant de définir des sous-expressions (opDefTable) en leur donnant un nom (nomReqCont) utilisable dans l'expression qui suit afin d'en faciliter l'écriture... et la lecture!

Au sein d'une requête, le plus souvent, moyennant quelques contorsions syntaxiques, il est possible d'utiliser une expression SELECT à la place d'une référence à une table. Assez rapidement, la lisibilité en souffre. L'énoncé WITH devient alors l'instrument privilégié de décomposition syntaxique (diviser pour régner). Le seul moment où il est nécessaire de recourir à l'emboîtement : lorsque le SELECT fait référence à une variable liée définie dans le contexte englobant.

Exemple — Université

- Quelles personnes étudiantes ne sont inscrites à aucune activité en 2013 ?
 - Quelles sont les notes inférieures à la moyenne pour l'activité IFT187 ?
- Donner le matricule, le nom de la personne, la note et la différence avec la moyenne.

```

with moy_ift187 as (
    select avg(note) as moy
    from Resultat
    where activite = 'IFT187'
),
res_ift187 as (
    select *
    from Resultat, moy_ift187
    where activite = 'IFT187'
)
select matricule, nom, note, (moy - note) as diff_moy
from res_ift187 join Etudiant using(matricule)
where note < moy;

```

UNIVERSITÉ

Étudiant clé (matricule)

matricule	nom	adresse
15113150	Paul	>Δ ^ς σ ^ς ᵇ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Activité clé (sigle)

sigle	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

TypeEvaluation clé(code)

code	description
IN	Examen intra
FI	Examen final
TP	Travail pratique
PR	Projet

Résultat clé(matricule, te, activite, trimestre)

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20132	80
15112354	FI	IFT187	20122	78
15113150	TP	IFT159	20132	75
15112354	FI	GMQ103	20122	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

Figure 1. Exemple — Université

Conclusion

Références

[Elmasri2016]

Ramez ELMASRI et Shamkant B. NAVATHE;
Fundamentals of database systems;
7th Edition, Pearson, Hoboken (NJ, US), 2016;
ISBN 978-0-13-397077-7.

[Date2015a]

Chris J. DATE;
SQL and relational theory: how to write accurate SQL code;
O'Reilly Media, 2015;
ISBN 978-1-4919-4117-1.

PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-09-29 00:13:07 UTC



Université de Sherbrooke